

```

❶ self.level_image = self.font.render(level_str, True,
    self.text_color, self.settings.bg_color)

    # Position the level below the score.
    self.level_rect = self.level_image.get_rect()
❷ self.level_rect.right = self.score_rect.right
❸ self.level_rect.top = self.score_rect.bottom + 10

```

The `prep_level()` method creates an image from the value stored in `stats.level` ❶ and sets the image's right attribute to match the score's right attribute ❷. It then sets the top attribute 10 pixels beneath the bottom of the score image to leave space between the score and the level ❸.

We also need to update `show_score()`:

```

scoreboard.py def show_score(self):
    """Draw scores and level to the screen."""
    self.screen.blit(self.score_image, self.score_rect)
    self.screen.blit(self.high_score_image, self.high_score_rect)
    self.screen.blit(self.level_image, self.level_rect)

```

This new line draws the level image to the screen.

We'll increment `stats.level` and update the level image in `_check_bullet_alien_collisions()`:

```

alien_invasion.py def _check_bullet_alien_collisions(self):
    --snip--
    if not self.aliens:
        # Destroy existing bullets and create new fleet.
        self.bullets.empty()
        self._create_fleet()
        self.settings.increase_speed()

        # Increase level.
        self.stats.level += 1
        self.sb.prep_level()

```

If a fleet is destroyed, we increment the value of `stats.level` and call `prep_level()` to make sure the new level displays correctly.

To ensure the level image updates properly at the start of a new game, we also call `prep_level()` when the player clicks the Play button:

```

alien_invasion.py def _check_play_button(self, mouse_pos):
    --snip--
    if button_clicked and not self.game_active:
        --snip--
        self.sb.prep_score()
        self.sb.prep_level()
        --snip--

```

We call `prep_level()` right after calling `prep_score()`.

Now you'll see how many levels you've completed, as shown in Figure 14-5.